PigPaxos: Devouring the Communication Bottlenecks in Distributed Consensus

Aleksey Charapko*
University of New Hampshire
Aleksey.Charapko@unh.edu

Ailidani Ailijiang
Microsoft
Ailidani.Ailijiang@microsoft.com

Murat Demirbas
University at Buffalo, SUNY
demirbas@buffalo.edu

ABSTRACT

Strongly consistent replication helps keep application logic simple and provides significant benefits for correctness and manageability. Unfortunately, the adoption of strongly-consistent replication protocols has been curbed due to their limited scalability and performance. To alleviate the leader bottleneck in strongly-consistent replication protocols, we introduce Pig, an in-protocol communication aggregation and piggybacking technique. Pig employs randomly selected nodes from follower subgroups to relay the leader's message to the rest of the followers in the subgroup, and to perform in-network aggregation of acknowledgments back from these followers. By randomly alternating the relay nodes across replication operations, Pig shields the relay nodes as well as the leader from becoming hotspots and improves throughput scalability.

We showcase Pig in the context of classical Paxos protocols employed for strongly consistent replication by many cloud computing services and databases. We implement and evaluate PigPaxos, in comparison to Paxos and EPaxos protocols under various workloads over clusters of size 5 to 25 nodes. We show that the aggregation at the relay has little latency overhead, and PigPaxos can provide more than 3 folds improved throughput over Paxos and EPaxos with little latency deterioration. We support our experimental observations with the analytical modeling of the bottlenecks and show that the communication bottlenecks are minimized when employing only one randomly rotating relay node.

ACM Reference Format:
Aleksey Charapko, Ailidani Ailijiang, and Murat Demirbas. 2021. PigPaxos: Devouring the Communication Bottlenecks in Distributed Consensus. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD '21), June 20–25, 2021, Virtual Event, China.* ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3448016.3452834

1 INTRODUCTION

Strongly consistent replication ensures that each operation accesses the latest committed database state, providing the illusion of a single copy centralized system [27]. This helps to keep application logic simple and provides significant benefits for correctness and manageability. Without strong consistency, the developers are forced to write a lot of peripheral code to handle side-effects and corner-cases arising from concurrent eventually-consistent operations. Moreover, an unanticipated combination of faults conspiring with the concurrency of operations is likely to leave those systems in states that are hard to debug and repair [24, 32, 37].

Unfortunately, the adoption of strongly-consistent replication protocols has been curbed due to their limited scalability and performance. In strongly consistent replication, all the communication drains through a single node – the primary (a.k.a. the leader), which constitutes a throughput bottleneck [2]. The leader performs a disproportionately large amount of work compared to its $N-1$ followers. For each consensus instance, the followers receive one message from the leader and send one message back. In contrast, the leader needs to send $N-1$ messages to the followers, and receive at least a quorum of messages back from these followers, not counting the interaction with clients. For instance, in a cluster of $N = 5$ nodes, the leader handles up to 4 times more messages than a follower. This puts a practical limit on the cluster size and prevents adding mode replicas to scale weaker-consistency reads or improve fault-tolerance. Therefore, scaling strongly consistent systems requires reducing the disproportionate load on the leader.

To address the single leader bottleneck and improve the throughput of strongly-consistent replication protocols, we introduce Pig, an in-protocol communication aggregation and piggybacking technique that aims to decouple the decision-making from the communication at the leader.

Pig replaces the direct communication between the leader and followers with a relay-/aggregate-based message flow. In particular, Pig employs randomly selected nodes from one or more follower subgroups to relay the leader's message to the rest of the followers in each subgroup, and to perform an aggregation of responses back from these followers. This overlay communication pattern allows the leader to only exchange messages with a small number of relay nodes. Moreover, the random alternation of the relay nodes across replication operations shields these relay nodes from becoming hotspots themselves: the extra traffic load a relay node incurs in one round is offset in consecutive rounds when the node no longer serves as a relay. These two properties combined improve the throughput scalability of the system.

Although aggregation-based approaches have been known and employed in the context of weak-consistency replication protocols [47], Pig demonstrates how they can be effectively applied to and integrated with strongly consistent consensus-based replication protocols. In particular, our experiments and analysis show that the rotation of the relay nodes in the Pig provides the most benefit for reducing the bottlenecks in strongly-consistent replicated state machines and improving throughput. Another novel and somewhat counterintuitive finding from our experiments and analysis is that

*Work partially completed at the University at Buffalo, SUNY.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SIGMOD '21, June 20–25, 2021, Virtual Event, China
© 2021 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-8343-1/21/06…$15.00
https://doi.org/10.1145/3448016.3452834

the communication bottlenecks are minimized when only one relay node is employed in contrast to what seems to be a more balanced configuration of using $\sqrt{N}$ relay nodes.

We showcase Pig in the context of classical Multi-Paxos [35] protocols, which, owing to their excellent fault-tolerance properties, drive state machine replication in many cloud computing services and databases [5, 9, 12, 16, 18, 20, 28, 30, 31, 34, 39, 45, 49, 52, 56, 57]. Despite significantly improving scalability and performance, PigPaxos reuses the same correctness proof as Paxos, because it preserves the core protocol, and modifies only the communication implementation to improve scalability and performance. As such, the communication piggybacking and aggregation technique used in Pig is applicable to many Paxos implementations [43, 44] and variants [3, 29]. Moreover, PigPaxos tolerates up to f node failures in a cluster of size $2f + 1$, just like the classical Paxos protocol. However, it can scale to large clusters with up to 25 nodes. Such large clusters may be useful in environments with more frequent failure rates to tolerate a greater absolute number of failures.

We implement PigPaxos in the Paxi framework [2] and evaluate many optimizations possible over Pig's basic dynamic communication overlay scheme. We also employ Paxi as a leveled playground to compare and benchmark Paxos, EPaxos, and PigPaxos in the context of the same in-memory key-value store implementation. Our experiments, conducted on AWS EC2 nodes in various LAN and WAN topologies, show that PigPaxos is surprisingly effective in scaling consensus to large clusters. For a 25 node deployment using a uniformly random workload in a 1000 item datastore, Paxos throughput gets saturated at 2000 req/sec, EPaxos throughput reaches its peak at 3000 requests per second, whereas PigPaxos scales beyond 7000 req/sec with little latency deterioration.

Our experiments also show that the aggregation has little overhead for the latency of PigPaxos as compared to the latency of Paxos. Another interesting result from our evaluation is that PigPaxos provides benefits over Paxos for consensus clusters as small as 5 nodes. In particular, due to aggregation at the relay nodes, PigPaxos throughput scales with respect to the size of the messages. This makes PigPaxos applicable for distributed database systems that use Paxos for replication such as CockroachDB [16] and Spanner [18]. PigPaxos also provides benefits for implementing geo-replicated distributed databases deployed over many regions around the globe, such as planetary-scale systems that have up to a few dozen nodes supporting a single partition [40].

We corroborate our experimental observations with the analytical modeling of the bottlenecks and show the extent to which PigPaxos helps shift these bottlenecks from the leader to the followers. We also provide formulas to estimate the relative load of different PigPaxos components in various configurations. For instance, we demonstrate that the cluster size has a limited and capped impact on follower load, while the number of relay groups significantly affects the leader load.

Paper outline. Section 2 provides a summary of Paxos. Section 3 presents the PigPaxos protocol, and Section 4 presents optimizations. Evaluation results are presented in Section 5, and discussion and detailed analysis of the findings are provided in Section 6. We discuss related work in Section 7, future work and extensions in Section 8, and conclude in Section 9.

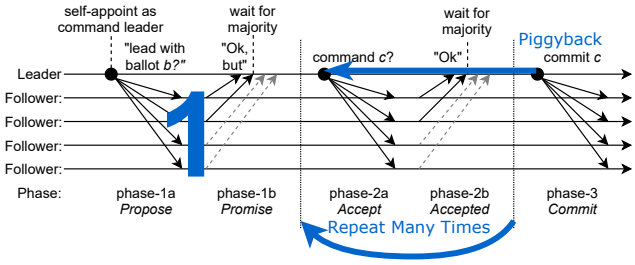


Figure 1: Multi-Paxos

2 BACKGROUND

2.1 Paxos Overview

The classical Paxos protocol [35] operates in three phases. The first phase, often called the propose phase, establishes a leader node. In this phase, a node tries to acquire leadership by reaching out to other nodes with some ballot number. The replicas ack the leadership proposal only if they have not seen a higher ballot. When a node collects a majority of acks, it can proceed to the second phase. In phase-2, the accept phase, the leader tells all the followers to accept a command. The command is either a new command of the leader's choosing or an old command if some nodes replied during phase-1 with an earlier uncommitted command. Once the leader receives a majority of acks from nodes accepting the command, the command is anchored, and the leader proceeds to the commit phase (phase-3). The leader then instructs the followers to finalize the command in the log and execute it against their state machines provided that there is no gap in their logs up to this command's slot.

This basic Paxos protocol is commonly extended to the Multi-Paxos [51] protocol, which allows the same leader to propose commands in subsequent slots (i.e., consensus instances) as illustrated in Figure 1. The leader election in phase-1 is performed only once, and this cost is avoided in subsequent consensus instances as long as the incumbent leader's ballot number remains the highest the followers have seen. Additionally, phase-3 is piggybacked to the next phase-2, further reducing the communication overheads. In the rest of the paper, when we mention the Paxos protocol, we refer to this Multi-Paxos optimization.

2.2 Leader Bottleneck

Paxos, like many other strongly consistent replication protocols, relies on a strong single leader to coordinate and order the operations in the system. This strong leader, however, is often a bottleneck, especially when every read and write operation has to go through it. The bottleneck has been intuitively known for quite some time, and many databases and storage systems [50, 53] purposefully picked a weaker consistency model to avoid a strong single leader and load balance themselves by shedding the workload equally among all the nodes. Some strongly consistent replication solutions [19, 41] also take a similar approach. For instance, EPaxos [41] avoids a single leader through the means of opportunistic, operation-level leadership. More recently, the intuition for the leader bottleneck was tested experimentally and analytically [2], and it was shown that the bottleneck occurs largely because of the extensive communication with the leader. A strong leader needs to send messages to

all the replicas, and receive responses to know when the operation has been successfully replicated in the state machine or log.

This fan-in/fan-out communication pattern is demanding for the leader. In the more common case, the leader will be bottlenecked at the CPU serializing, deserializing, and processing these messages. For example, in our testing, we have experimentally verified that serializing and deserializing just around 100,000 phase-2a and phase-2b Paxos messages are enough to saturate one CPU core of our test machine. However, the node also needs to process these messages, further consuming limited CPU cycles. In some rare cases, when the messages are very large, the bottleneck may shift towards the available network bandwidth. However, in both situations the root cause of the problem is the same – too many messages are sent, received, and processed by one node.

3 PIGPAXOS

3.1 Pig Primitive

Strongly consistent replication protocols, such as Paxos and primary-backup protocols, exhibit similar communication patterns. They rely on a single stable leader used for ingesting and ordering the operation. The leader replicates the operations along with their order positions to the follower nodes, and the followers reply to the leader to acknowledge the operations. We can break down this communication into two steps: fan-out message flow from the leader, and fan-in flow to the leader.

The major problem with both fan-out and fan-in communication flows is that they disproportionally bog down the node initiating the communication – the node needs to send messages to $N-1$ other nodes, where N is a total number of machines/processes, and then receive $N-1$ distinct replies. This communication pattern is the source of the leader-bottleneck in replicated state machines [2]. Pig is a simple primitive designed to solve this problem and improve the efficiency of fan-out and fan-in message flows. Pig builds on top of a multicast idea to disseminate the messages from one source to multiple recipients efficiently. Unlike multicast, Pig assumes two-way communication and aggregates the responses coming back from the recipients.

Pig eschews direct communication between the initiating and replying nodes. Instead, the communication initiator picks another random node as the relay and sends the message to it. The job of a relay is threefold. Firstly, it acts as a regular node in the system and processes the incoming message. Secondly, it re-transmits the message over to the remaining nodes. Thirdly, it acts as an aggregator for the replies on the way back to the initiator.

The later aspect of Pig is the major difference from a peer-to-peer multicast. When nodes reply to the message, they reply to the relay instead of the communication initiator. The relay is then aggregating the replies together and piggybacks them into one concise message to the initiator. We illustrate the Pig communication pattern in Figure 2

At first glance, Pig may appear as inefficient as a regular fan-out/fan-in round trip, since now a relay node is doing all the heavy work instead of the initiator. However, as we repeat Pig rounds, we pick the relay nodes uniformly randomly, amortizing the extra work done by the relay over many rounds in which a node does

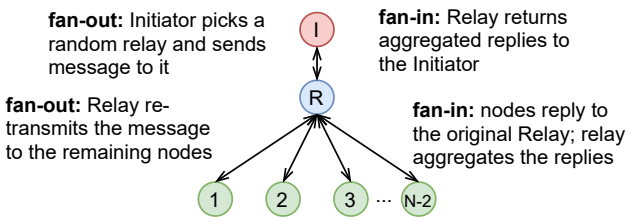


Figure 2: Pig communication pattern for a single fan-out & fan-in pair.

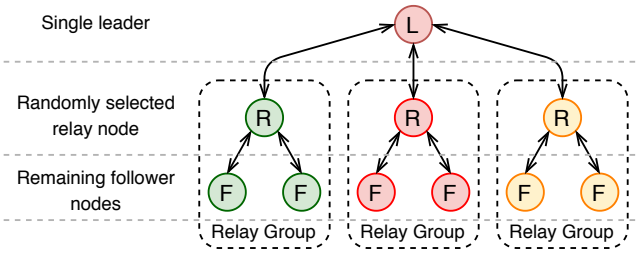


Figure 3: PigPaxos communication flow. All nodes are divided into groups, and the leader communicates only with a random node from each group. The nodes contacted by the leader relay the messages to their peers within the group and aggregate their responses to reply back to the leader.

not serve as a relay. This approach works well when many fan-out/fan-in communication cycles can overlap similar to pipelining. A node may serve as a relay in the communication round i, and as a follower in other largely concurrent rounds $i+1, \ldots, i+n$.

Our implementation of Pig hides behind two general APIs: `broadcast(broadcastMsg)` and `send(sendMsg, toNode)`. The `broadcast(broadcastMsg)` API sends the message `broadcastMsg` from the initiator to all the nodes participating in the current message exchange round. The `broadcastMsg` must contain a `PigMsgID` to uniquely identify the Pig round. Every node, including the relay, replies to the relay node using the `send(sendMsg, toNode)` API. When replying, `sendMsg` must contain the same `PigMsgID` as the original fan-out message. The relay uses this `PigMsgID` to match the reply with the original fan-out message and aggregate all relevant replies together. Once the relay receives the required threshold of replies (or it times out), it forwards the aggregated reply to the initiator, completing the Pig communication pattern.

3.2 Putting Pig into Paxos

Leveraging our observations on message exchange in Paxos, we can apply our Pig communication flow to the Paxos-family of replicated state machines. We can use one or more instances of Pig in a Paxos deployment to shard both the fan-in and fan-out communication across the nodes. Figure 3 illustrates how multiple Pig instances can be overlaid in the Paxos cluster. Initially, all nodes are divided into several distinct non-overlapping relay groups. This grouping may happen with the help of a hash function or be pre-defined based on the node topology. For example, in a geo-distributed setup, we may statically define the groups based on the regions in which

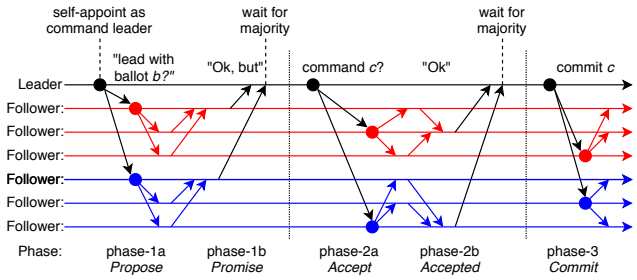


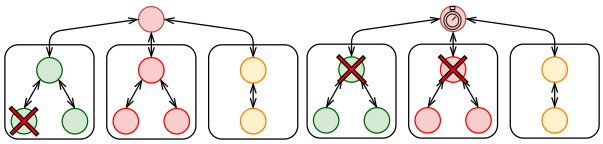
Figure 4: The three phases of PigPaxos with a single level communication relay.

nodes are located. Our reference implementation uses such static assignment of nodes to groups. Within each group, we use Pig for communication from the leader to the followers in the group.

In Figure 4 we demonstrate the Pig communication flow of PigPaxos for Phase-1, Phase-2, and Phase-3. Note that, using Multi-Paxos optimization with a stable leader, PigPaxos would only perform Phase-2 for each consensus instance, with Phase-3 messages piggybacked to Phase-2. We break-down and describe the communication flow consisting of a fan-out and fan-in as follows.

(1) When the leader initiates a fan-out communication, it starts the Pig pattern for each relay group by picking a random relay from every group. The random rotation of relay nodes balances the communication burden among the followers. As relays alternate, the extra load each node experiences in the relay round gets offset in other concurrent but consecutively numbered rounds in which that node ceases to be a relay.
(2) Upon receiving a message from the leader, a relay node processes this message as a regular follower and resends the message to the remaining nodes in its relay group.
(3) Upon receiving the messages, the followers start the fan-in pattern by responding back to the relay node as if they were responding directly to the leader.
(4) Each relay node waits for the followers' responses and piggybacks them together into a single message. By default, the relays wait for all followers in the group to respond. Since such unbounded wait may create performance problems when some follower becomes sluggish or fail, PigPaxos employs a tight timeout at the relay nodes. If some followers do not reply within the timeout, a relay stops waiting and replies to the leader with all responses collected so far.
(5) The leader performs the final aggregation of replies from relays and decides whether the quorum of votes has been received.

In some cases, we can break the communication pattern short, when it is obvious that no further progress can be made. Usually, these short-cuts arise during a leader change. For example, when a relay node learns of a new leader with a higher ballot either through a direct message or a reply from a follower, it no longer tries to finish the aggregation, and replies to the old leader with a reject message carrying the new ballot.



(a) Follower failure causes the relay node to timeout before responding to the leader.

(b) Relay failures break message delivery, causing leader to retry with different relays.

Figure 5: Failure scenarios at follower and relay nodes.

3.3 Paxos to PigPaxos mapping

PigPaxos generalizes the communication of the Paxos protocol. Paxos has $N-1$ relay groups, where each group has one element. In contrast in PigPaxos there are p groups where $p \in \{1..N-1\}$, and each group is a set of up to $N-1$ nodes. The cardinality of the union of all groups is $N-1$. This formulation allows the groups to overlap, which may have advantages in terms of adding redundant channels to reach some nodes. However, for simplicity's sake, we preclude intersecting groups in the rest of the paper.

We note that the safety proofs of Paxos [35] do not depend on the communication implementation between the leader and follower nodes. In Paxos, maintaining correctness in spite of failures is guaranteed by quorum size and the information exchanged within the quorums, and the proofs are oblivious to how the communication of this information is achieved. Therefore, PigPaxos preserves the safety properties of Paxos, as it only modifies the communication implementation. For reasoning about liveness, the message flow pattern and the use of relay nodes requires special attention, as a failure of a relay node has a disproportional impact compared to the failure of a regular follower. Similar to Paxos, PigPaxos requires some level of synchrony to make progress in a fault-tolerant manner, as FLP impossibility prevents consensus in fully asynchronous networks with machine failures [21]. PigPaxos assumes some upper bound delay on message propagation and retries if communication takes longer. This ensures that when the synchrony is restored to the adequate levels as prescribed by the upper bound delay, the protocol will make progress. Since PigPaxos uses a uniformly random selection of relay/aggregator nodes at each round, it circumvents the problem of having some minority failures denying all progress.

3.4 Fault Tolerance

Similar to Paxos, PigPaxos considers a crash failure model in which nodes may silently crash. Since PigPaxos implements an unchanged Paxos protocol, it tolerates up to f node failures in a cluster of size $2f+1$. However, Pig communication flow introduces its failures into the PigPaxos, potentially causing slightly different performance behavior than classical Paxos implementation, as faults may cause the retry of Pig instances and delay the message propagation. We distinguish two common communication failure cases: a follower failure and a relay node failure.

PigPaxos deals with follower failures by enacting a tight timeout at the relay node – the relay does not wait for responses indefinitely and replies to the leader either upon collection of all children responses or reaching a timeout T_r. The relay timeout procedure allows a partial set of responses to reach the leader in the hopes

that a majority is still satisfied. We illustrate this type of failure in Figure 5a. In many cases, the leader remains unaffected by the timeout of a minority of relay groups, as the majority quorum may be reached by votes from other relay groups.

Relay node failures are more severe. When a relay fails, the leader will receive no feedback/votes from that entire relay group. If a sufficient number of votes are tallied with other relay groups, the protocol will proceed as normal, however, if the leader cannot get the majority quorum, it will timeout. Figure 5b demonstrates this failure scenario.

Regardless of the failure type, a healthy PigPaxos leader institutes a retry upon reaching the timeout. A retry uses a new, randomly selected relay group to ensure that a few specific failures cannot bring the system down. Under severe failures when close to f nodes fail, the retries may become more frequent, since it may take multiple attempts to randomly find a Pig instance with healthy relays to collect enough responses. However, this does not cause availability loss, as the system remains live and will make progress, albeit at a reduced rate.

The two distinct timeouts, one for the relay nodes and one for the leader, have a strict relation, as the relay node timeout needs to be smaller than the leader timeout to allow the relays to deliver the partial response to the leader before the leader reaches its timeout. The timeout retries do not affect the safety of the protocol, as they only hinder message delivery and do not affect decision-making at the leader. Retrying a phase-2 message cannot cause any problems in PigPaxos, since the leader keeps track of all nodes that have voted on the slot in a given Paxos ballot. If a duplicate vote arrives, the leader will recognize it and not count the vote twice. Furthermore, the leadership itself is protected by the ballot number, allowing a different node to use regular Paxos means of overtaking the leadership as needed. As a result, if some new leader is in a better position to communicate with the cluster, it can get elected with a higher ballot and proceed, while resend attempts by the old leader will fail as having an obsolete ballot.

4 PIGPAXOS OPTIMIZATIONS

4.1 Partial Response Collection

In the basic PigPaxos protocol, the relay nodes wait to hear from all nodes in their relay groups before aggregating and forwarding these messages to the leader. This may cause slowdowns when some nodes are sluggish or have crashed. One way to mitigate the problem is to avoid waiting for all followers in the group to reply, and send a partial response to the leader when some threshold g_i of nodes in a group i have responded. We define g_i in terms of the number of nodes (PRC) we are willing to omit in the aggregation: $g_i = n_i - PRC$, where n_i is the size of the group i. It is still important that the leader collects at least a majority votes across all relay groups, so the threshold g_i (or rather the number of slacking nodes PRC), must be chosen accordingly: $\sum_{i=1}^{R} g_i >= \lfloor \frac{N}{2} \rfloor + 1$, where R is number of relay groups and N is the total number of nodes. This will improve the latency of PigPaxos as well as reduce momentary slowdowns due to some sluggish or failed nodes. However, this may also cause issues with more severe failures – a crashed relay group and low g_i (high PRC) may cause the leader to not receive enough replies, requiring more communication to learn the missing votes.

4.2 Gray List of Potential Failures

Selecting relay nodes from a uniformly random distribution is an easy strategy that provides liveness even under severe failures with enough retries. However, even a small number of failures may negatively impact the latency of the system in some configurations. For example, consider two relay groups setup in the $2f + 1$ cluster. One group will consist of $f + 1$ nodes, while the other group is a minority one with just f machines. In such a configuration, even a single failure in the majority group can bring adverse performance effects – when a failed node is selected as a relay, it will cause the entire relay group to time-out, increasing the operation's latency.

A simple solution to combat the problem is to keep track of such failed, slow, or potentially problematic nodes. Each node maintains a gray list of potentially problematic nodes, such as nodes that have had network failure. The leader, upon selecting a random relay, can exclude the gray-listed nodes from the relay consideration for some time. However, gray-listed nodes may still be picked as relays on occasion as a way to probe their health.

4.3 Special Case of Single Relay Group

PigPaxos does not restrict the number of relay groups. As we showed earlier, when $R = N - 1$, PigPaxos becomes a normal Multi-Paxos system. Another special relay group configuration is $R = 1$. In this setup, the leader needs to communicate with just a single relay node, potentially removing as much work from the leader as possible. However, waiting for all nodes to reply to the relay in $R = 1$ becomes problematic in case of slow or failed nodes. Configurations with $R > 1$ often mask one slow relay group by getting a majority of votes from other groups. Fortunately, a single relay group provides a relay node with a nearly-global cluster view, allowing it to use *partial response collection* more efficiently. When the majority of nodes have replied to the relay, it can safely respond to the leader without waiting for the minority of slow nodes. This is not possible with more than one relay group, as each relay node has only partial information about all replied nodes, and cannot make a decision on whether the global majority has been reached.

5 PERFORMANCE EVALUATION

5.1 Implementation & Testbed

We implemented PigPaxos in Go using Paxi [2]. Paxi is an open-source testbed for prototyping, evaluating, and benchmarking consensus and replication protocols. It provides most of the functionality that any coordination protocol needs, including network communication, the state machine of a key-value store, and multiple types of quorum systems. Paxi also includes a built-in YCSB-like [17] benchmark for testing the protocols implemented in a framework.

Our implementation of PigPaxos in the Paxi framework required almost no changes to the core Paxos code and focused only on the message-passing layer and relay group orchestration. The entire protocol was implemented in just 1200 lines of code and is available as opensource at https://github.com/pigpaxos/pigpaxos. The performance of Paxi's Paxos implementation is on par with industry implementations [2]. We conducted our experiments on AWS EC2 m5a.large instances with 2 vCPUs and 8 GiB of RAM. To ensure

that the client performance does not impact the results, we used the larger m5a.xlarge instances with 4 vCPUs for the clients.

We use closed-loop or synchronous clients, meaning that each client can have only one outstanding operation and will not start a new operation before the completion of the previous one. To drive the throughput in the system we used up to 150 clients, better approximating a real environment with many clients interacting with the system. We measure latency at the client, meaning that the operation latency is the time between issuing a request by the client and receiving a successful response.

The replication protocols implemented in Paxi use an internal in-memory key-value store for testing purposes. Paxi's benchmark utility is capable of generating YCSB workloads against this internal datastore. It provides a fair comparison as the only components that change under test are replication protocols. For our Multi-Paxos and PigPaxos evaluation, we used a workload with 1000 distinct keys and a uniform probability of selecting each key. It is worth noting that systems driven by a single Multi-Paxos instance do not experience any performance variation due to the workload skew, since all operations are placed in the same single write-ahead log. However, leaderless protocols, such as EPaxos [41] do experience conflicts due to the workload skewness. Using 1000 keys allowed us to adjust the workload for EPaxos to avoid any conflicts and present the best possible leaderless scenario.

We used a KV-pair with small keys and values of 8 bytes in most tests, however, in select experiments we increased the size of the value up to 1.25 Kb. We used an even distribution of reads and writes, since the generic implementation of Paxos does not differentiate between different operation types, and both reads and writes must go through the replication (phase-2) of Paxos. Unless otherwise stated, we used a partial response collection strategy and did not wait for the slowest node in each group.

5.2 Number of Relay Groups

The performance of PigPaxos is controlled mainly by the number of relay groups used for message dissemination and aggregation. A basic strategy for determining the number of relay groups in a scale-free manner is to use $\sqrt{N}$ as the number of groups. For example, $N = 25$ implies using 5 groups with 5 nodes each. This makes the leader communicate with 5 relay nodes and the relay nodes to talk to 4 of their peers, a seemingly load-balanced setup. However, this naive strategy works well if the relays are static. PigPaxos, however, uses a uniformly random relay rotation and requires a different strategy for computing the best number of relay groups. Figure 6 we show the maximum throughput of a 25-node PigPaxos cluster with and without the random relay node rotation. Please note that the protocol without random relay rotation does not have the same liveness properties.

As expected, using $\sqrt{N}$ as a proxy for the number of groups works well when the relay nodes are static and do not change. However, the best throughput is achieved with random relay node rotation and the smallest number of relay groups possible. Such a setup minimizes the communication bottleneck at the leader the most and shifts a large portion of the message exchange overheads uniformly across the followers. The relay nodes in a single relay group setup do not get overwhelmed when talking to 23 other

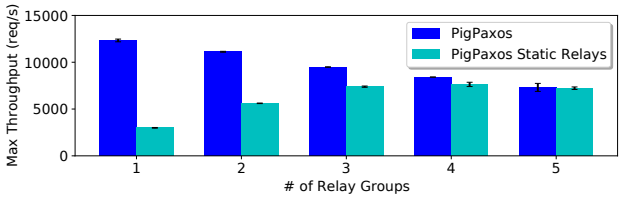


Figure 6: Number of relay groups on 25-node PigPaxos with single relay layer. Static relays represent PigPaxos with non-random, predifined relay in each group.

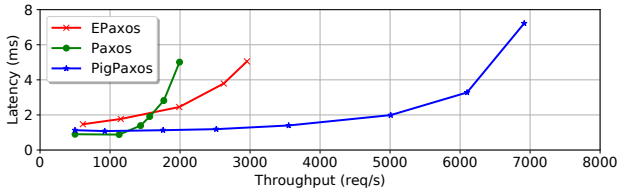


Figure 7: Latency and Throughput on 25-node cluster with 3 relay groups.

nodes because the relays are rotated at every round. As relay nodes randomly rotate, the extra load in one round is offset by other concurrent rounds in which the node no longer serves as a relay. We discuss and analyze this observation further in Section 6.1.

5.3 Latency and Throughput

Using relays adds a latency overhead. To study the impact the relay nodes have on PigPaxos latency, we compare PigPaxos latency with that of a corresponding traditional Paxos deployment. We also compare with Egalitarian Paxos (EPaxos) [41], since it presents an alternative way of reducing the leader bottleneck – eliminating dedicated leaders altogether. We run EPaxos with no conflicting operations to make sure we capture its best possible performance. The benchmark clients communicate with Paxos and PigPaxos leaders for all operations, and with a random node in EPaxos on every operation.

Figure 7 illustrates the latency and throughput performance of these three protocols in a 25-node cluster. EPaxos naturally shows better scalability than Paxos since it allows all nodes to become leaders and serve operations. This removes the single leader bottleneck, giving EPaxos 50% better throughput in conflict-free cases. On the other hand, EPaxos becomes limited by its increased communication costs. Since it needs to use larger fast quorums to commit operations, it incurs more communication and higher latency to reach consensus. The message size complexity of EPaxos also depends on the cluster size N, as the protocols need to keep track of dependencies coming from each node. This reliance on the cluster size for message handling reduces the scalability of EPaxos as N grows. For instance, we have conducted a small experiment to see the overhead of more nodes in EPaxos and found that messages for a 25 nodes cluster take roughly 4 times more time to serialize than messages for a 5-node cluster. Furthermore, we have estimated

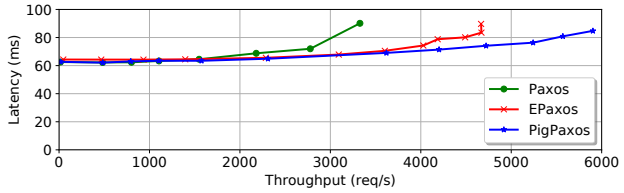


Figure 8: Latency and Throughput on 15-node WAN cluster in Virginia, California and Oregon.

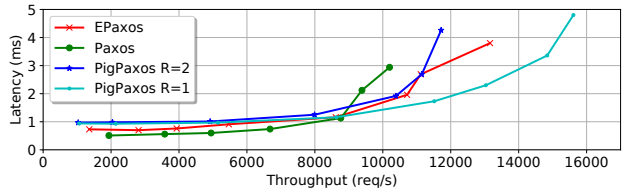


Figure 9: Latency and Throughput on 5-node cluster with 2 relay nodes.

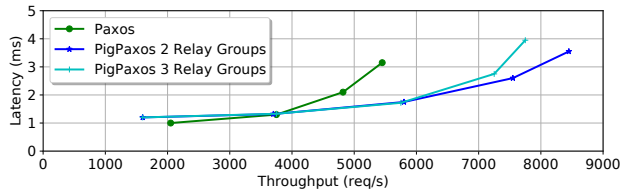
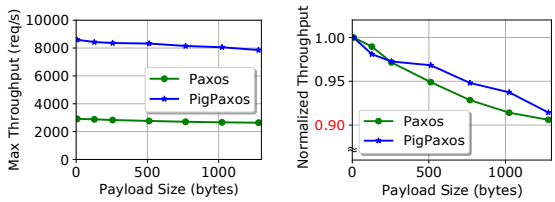


Figure 10: Latency and Throughput on 9-node cluster with 2 and 3 relay groups.



(a) Maximum throughput. **(b) Normalized throughput.**

Figure 11: Performance of PigPaxos at various payload sizes, varying from 8 to 1280 bytes. In (b), note that the throughput scale starts at 0.90 of maximum throughput.

that PigPaxos' aggregated *P2b* serializes about 14% and deserializes about 8% quicker than the similar *PreAcceptReply* message of EPaxos. Other message types have even bigger serialization performance differences, making every EPaxos node spend more time in message handling than the PigPaxos leader.

Paxos, on the other hand, is limited by a single leader exchanging messages with all the followers. Initially, Paxos has a lower request latency, but its throughput gets saturated quickly and reaches its limit of around 2000 requests per second. While PigPaxos pays the initial price of having higher latency, we see that it scales to a much higher throughput with little latency deterioration.

PigPaxos provides great throughput improvements over traditional Paxos in wide-area deployments as well. In Figure 8 we illustrate PigPaxos in 15 nodes configuration spread over AWS Virginia, California and Oregon regions. Each region represents a separate PigPaxos relay group, with the leader node located in the Virginia region. We deployed the clients in the same regions as the leader node. For EPaxos, we deployed clients in all regions and configured them to only communicate with nodes in their local regions. In WAN, the latency is dominated by cross-region distance, and the difference between Paxos and PigPaxos is not observable at low loads. EPaxos latency is slightly higher than both Paxos and PigPaxos due to a larger quorum and geographical properties – depending on the region of request origin, the distance to reach quorum differs by approximately 5 ms in this setup. Similar to the local area experiments, PigPaxos maintains low latency for much higher levels of throughput, and out scales both Paxos and EPaxos.

5.4 Small Clusters

We find that the benefits from PigPaxos extend to small clusters as well. In Figure 9 we show a 5-node cluster running Paxos and PigPaxos with 1 and 2 relay groups ($R = 1$ and $R = 2$). For the $R = 1$ case, we used the single relay node optimization to wait for only the majority of nodes to aggregate at relay before retransmitting back to the leader. The difference in latency between the Paxos and PigPaxos protocols is more pronounced in this setup, as Paxos can maintain its low-latency performance for longer. However, even in this smaller cluster, PigPaxos scales to higher throughput than Paxos since the leader communicates with fever nodes.

In this small cluster, EPaxos shows its full potential, as it is not yet bottlenecked by expensive messages and can take advantage of its opportunistic leadership of operations. EPaxos shows around 12% higher maximum throughput than PigPaxos for $N = 5$ with $R = 2$. A single relay group PigPaxos configuration ($R = 1$) still outperforms all protocols, since it can spread the load in the cluster nearly evenly between all nodes without the penalty of conflict detection, resolution, and high message complexity.

We repeat the experiment on Paxos and PigPaxos in a slightly larger 9 nodes cluster with $R = 2$ and $R = 3$ configurations, as shown in Figure 10. This setup is still relatively small but allows us to use higher values of R. As before, PigPaxos scales better than Paxos in both 2 and 3 relay groups deployments. However, with a bigger cluster size, Paxos's latency advantage over PigPaxos diminishes quickly even at lower throughput levels.

5.5 Payload Size

Payload size may have an impact on the communication performance of the system. Large messages require both more resources for serialization and more network capacity for transmission. To study how different payload size impacts the performance of PigPaxos and Paxos, we experiment with 25-node clusters, where PigPaxos uses 3 relay groups. To gauge the performance and scalability with respect to payload size we measured the maximum throughput on each system under a write-only workload generated by 150 clients running on 3 VMs.

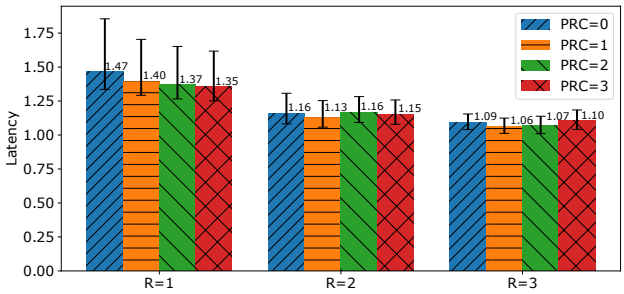


Figure 12: Median and IQR latency with different configurations of partial response collection with different numbers of relay groups in a 25 nodes cluster. $PRC = x$ indicates that the relay is not waiting on x slowest replicas to send an aggregated response to the leader.

Figure 11a shows the maximum throughput of PigPaxos and Paxos at payload sizes varying from 8 to 1280 bytes. While PigPaxos show three times more throughput than Paxos at all payload sizes, both protocols exhibit a similar relative level of degradation as the payload size increases. Figure 11b illustrates the throughput normalized to the maximum observed value.

5.6 Partial Response Collecton in Steady-State

Partial response collection optimization allows a relay group to respond to the leader without waiting on one or more of the slowest nodes, slightly improving the latency in the steady operation and allowing to better tolerate occasional straggler nodes. In Figure 12 we illustrate the impact of this optimization on median and IQR (25th and 75th percentile) latency. For this experiment, we ran the benchmark at a fixed workload of 3000 requests/second over a 300 seconds interval. We see the real benefit from this feature in the steady-state in a configuration with just a single relay group. In such a setup, a relay needs to wait on 23 other nodes before replying, making an occasional straggler slow down the system In the configurations with multiple relay groups, it is sufficient to have an entire relay group to not reply and the leader still reaching the majority, making the benefit of masking a slowdown in a few occasional nodes not needed when a slowdown of an enitre relay group can be tolerated.

5.7 Performance Under Failures

As discussed in Section 3.4, PigPaxos remains live for as long as the majority of nodes as still up. However, the Pig communication adds additional failure modes that can impact the performance of the protocol, as relay failures or many node failures may cause timeouts at both the relays and the leader. This is evidenced in Figure 13 in configuration with $R = 2$ and no partial response collection without the gray lists. In this figure, we show median latency from a steady workload generated by 30 clients in a 25 nodes cluster with a single failed node in the group constituting the majority of nodes.

In this experiment, a healthy group of 11 nodes and a leader do not have sufficient nodes to get to a majority. As a result, with no partial response collection, the leader must wait for the second group to timeout and return a partial response. Moreover, if the

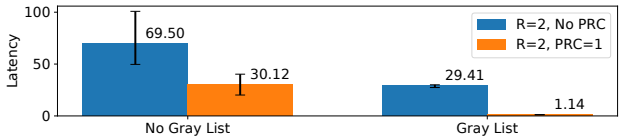


Figure 13: Median and IQR latency using gray lists at different levels of partial response collection in a 25 nodes cluster with 2 relays groups($R = 2$) and one node failure. $PRC = x$ indicates that the relay is not waiting on x slowest replicas to send an aggregated response to the leader.

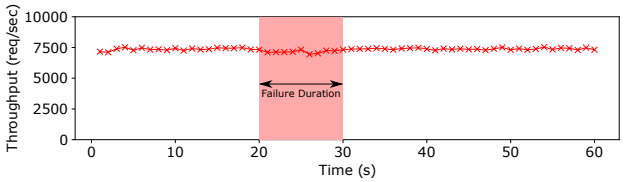


Figure 14: Maximum throughput on 25 nodes cluster with 3 relay groups under a single group failure. Throughput is sampled over 1-second intervals.

faulty node is selected as the relay, then the leader needs to timeout itself and retry, causing an elevated latency even with partial response collection turned on. Enabling leader gray lists (Section 4.2) allows the leader to stop assigning the failed node as a relay as frequently, improving the latency further.

Figure 14 illustrates how the maximum throughput of PigPaxos is impacted when one of the 3 relay groups in a 25 node cluster is faulty with multiple nodes not responding. In this experiment, we did not use any additional optimizations beyond the group timeout to illustrate the worst possible case of a fault relay group. The timeout was set to 50ms, more than 40 times over the normal operation latency.

A failure in a single relay group causes the relay group to timeout. However, the two other relay groups still constitute the majority of nodes and can respond to the leader quickly, resulting in very little change in performance. We find that the maximum throughput over the duration of the fault declined by only 3%, mainly due to the leader having to wait a little longer.

6 DISCUSSION

6.1 The Leader Bottleneck and Relay Groups

PigPaxos relies on the relay groups to deliver messages to every participating node. As we have observed in Figure 6, the number of relay groups impacts the performance significantly. For example, with two relay groups ($R = 2$), PigPaxos achieved 11% higher throughput than the $R = 3$ configuration. To explain this result, we refer to the performance modeling ideas in Ailijiang et al. [2] and extend to cover PigPaxos. We adapt the model to look at the number of exchanged messages M in every node of the PigPaxos protocol as a proxy for the load on the node. A leader needs to receive one message from the client and eventually send a reply, besides, it communicates with R relay groups. The total number of

# of Relay Groups (R)	Messages at Leader (M_l)	Messages at Follower (M_f)	M_l/M_f
1	4	3.92	1.020
2	6	3.83	1.567
3	8	3.75	2.132
4	10	3.67	2.725
5	12	3.58	3.352
24 (Paxos)	**50**	**2**	**25.0**

Table 1: Message load at leader and followers for different number of relay groups in 25 nodes cluster.

messages M_l handled by the leader is then expressed as:

$$M_l = 2R + 2 \tag{1}$$

The message load on the followers depends on the role it performs for a given round, as relays handle more messages than the regular followers. A non-relay processes only one round trip communication with the relay node. A relay node, on the other hand, needs to handle one round trip message exchange with the leader and every remaining follower in the relay group. However, the random alternation of the relays shields them from becoming hotspots, since the extra traffic load a relay node incurs in one round is offset in consecutively-numbered concurrent rounds when the node no longer serves as a relay. Therefore, on average, we have the following message load M_f on each follower:

$$M_f = 2\left(\frac{R}{N-1}\right)\left(\frac{N-R-1}{R}\right) + 2 \tag{2}$$

$$= \frac{2\,(N-R-1)}{N-1} + 2 \tag{3}$$

where N is the total number of nodes in the system, and $\frac{R}{N-1}$ is the probability of a node being chosen as a relay for the round.

This analysis of the number of message exchanges at the nodes paints a similar picture to our empirical results in Section 5.2, indicating that a small number of relay groups R eliminates the leader bottleneck more effectively. The reason for this is that the leader still handles more communication and data processing than the "average" follower node. In Table 1 we show the number of messages handled by the leader and followers for a different number of relay groups in a 25-node cluster. We also compute the ratio of messages processed by the leader to that of the followers. We observe a dramatic increase in this ratio as the number of relay groups increases, indicating that the load on the leader and the followers becomes more disproportionate with additional relay groups. Such disbalance reaches a factor of 25 times for a regular Multi-Paxos deployment. We have confirmed this growing disparity between load at the leader and followers by measuring the CPU utilization and found a widening difference in CPU usage between the leader and the followers as the number of relay groups increases.

We have verified the message load experimentally using Retroscope [15] by collecting the in-flight message information. Figure 15 illustrates a heatmap of in-flight messages to and from each node in a 9-server AWS cluster under a 1500 req/s workload. In the figure, the color depth represents the communication intensity, with the maximum message exchange rate clocked at 33 messages in one millisecond. We observe that PigPaxos distributes the communication load in the cluster better compared to Multi-Paxos, even though the leader still remains a communication bottleneck.

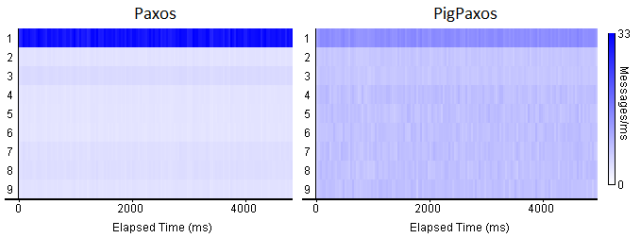


Figure 15: Heatmap of in-flight messages between nodes in 9 nodes Paxos and PigPaxos cluster on AWS EC2 under 1500 requests per second workload. Node 1 is a leader in both cases. PigPaxos used 3 relay groups. Every in-flight message is counted upon the first observation in Retroscope.

6.2 Single Relay Group Configuration

As evidenced from our evaluation (Figures 6 & 9), single relay groups perform admirably well in PigPaxos, and one may ask a question of whether any other configurations are required at all. One drawback of $R = 1$ PigPaxos is the potential for slightly higher tail latency under failures. When the leader picks a crashed or slow node as a relay, it will stall the request for the timeout duration, increasing the tail latency. At the same time, $R > 1$ configurations often reach the majority with one relay group not replying at all, seamlessly masking such failures. However, under more severe failure conditions, timeouts can occur even at $R > 1$.

Another area where $R > 1$ PigPaxos shines is geo-distributed replicated state machines. The ability to assign all nodes in the region to a single relay group allows the PigPaxos leader to send only one message per region across the wide-area network (WAN). With many cloud providers charging for WAN traffic, a PigPaxos system can provide significant savings to applications requiring geo-redundancy and strong consistency. For example, a 3 region deployment with 3 nodes in each region will have only 2 messages going across WAN for each write operation. A Paxos-backed system, in contrast, will send 6 separate messages, a 3-fold increase in WAN traffic. Having a relay group in each data center also means that the majority quorum is reached by nodes in the data centers closest to the leader, while $R = 1$ setup may rotate the relay node to a far away datacenter causing higher latency.

6.3 PigPaxos in Small Clusters

PigPaxos scalability benefits apply not only to large clusters but also to smaller deployments. As we illustrate in Figure 10, a 9 node PigPaxos cluster with $R = 2$ improves the throughput over classical Multi-Paxos by as much as 57% with very little degradation in latency, and even more, performance is possible with $R = 1$. Moreover, the benefit of PigPaxos extends down to 7 and even 5 node clusters (Figure 9). In the latter case, PigPaxos has very little flexibility for its configuration, as the only options are to use one or two relay groups. However, in both situations, we have seen performance improvements compared to the regular Multi-Paxos. Using the analytical approach described earlier, Table 2 illustrates the relative message-induced bottleneck of Paxos and PigPaxos leaders in a small 5 nodes cluster.

# of Relay Groups (R)	Messages at Leader ($M - l$)	Messages at Follower (M_f)	M_l/M_f
1	4	3.5	1.142
2	6	3	2.0
4 (Paxos)	10	2	5.0

Table 2: Message load at leader and followers for different number of relay groups in a small 5 nodes cluster.

6.4 PigPaxos Overheads

Pig protocol aims to shift the load in fan-out/fan-in concurrent communication away from a node that initiates communication. PigPaxos adopts this pattern to reduce the leader bottlenecks and even out the costs of message handling/processing in the entire cluster. However, in the process, Pig introduces several overheads and performance compromises. Most importantly, Pig splits a direct communication between the leader and the followers into a two-hop message exchange, leading to a latency increase. For example, in Figures 7, 10, and 9 we can observe Multi-Paxos having as much as 40% lower latency when the workload's throughput is low.

Another important consideration is the message size in Pig. While the leader in PigPaxos processes fewer messages, the aggregated responses are typically bigger than the individual responses in Multi-Paxos, erasing some of the Pig benefits. However, in PigPaxos we make sure that the aggregated message is as small as possible by removing all redundant information. For instance, Paxos leader keeps track of all followers who voted to prevent double-counting. However, instead of including the ids of all votes in the aggregate message, relays include ids of missing/non-voted nodes. The missing ids set is much smaller, but sufficient for the leader to figure out the correct votes and prevent double-voting. Finally, it is worth noting that despite changing how messages flow in the protocol, PigPaxos does not increase the overall number of messages exchanged between nodes.

6.5 Number of Relay Layers

We intentionally limited our discussion to a single layer of relay nodes. However, Pig communication flow can be extended to multiple layers of relay nodes. This can help if the first layer of relays becomes the bottleneck, as deeper relay layers can offload some of the communication burdens from prior layers.

Intuitively, it is hard to shift the bottleneck to relays, since this means that every non-leader node needs to do more work than the leader. We show that even in the most favorable configurations, the leader remains a bottleneck. The leader's message load M_l is a linear function of the number of relay groups R, as shown in formula 1. To minimize the leader message load, we need to minimize the number of relay groups, which cannot be smaller than 1. As a result, the smallest message load on the leader is $M_l = 4$. At the same time, with $R = 1$ and $N \rightarrow \infty$, the load on the followers $\frac{2(N-R-1)}{N-1} + 2$ asymptotically increases to 4, as $\lim_{N\rightarrow\infty} \frac{2(N-2)}{N-1} + 2 = 4$. In other words, in the most favorable case, the average/amortized load on follower nodes only matches that of the leader, suggesting that the leader remains a bottleneck regardless of the cluster size and the number of relay groups. Since the bottleneck cannot be shifted entirely to the followers with any N, adding more layers to offload the communication work from the followers will not result in a performance gain.

7 RELATED WORK

7.1 Consensus Protocols

Scalability bottlenecks have been a big problem in consensus protocols and have received a lot of attention. Flexible quorums Paxos, or FPaxos [29], observes that the majority quorum is not necessary for phase-1 and phase-2. Instead, it shows that the Paxos properties hold as long as all quorums in phase-1 intersect with all quorums in phase-2. This result enables deploying Multi-Paxos protocols with a smaller phase-2 quorum, providing better performance at the cost of reduced fault tolerance.

While using smaller phase-2 quorums is beneficial for reducing latency, it does not reduce the leader bottleneck and does not help improving scalability or throughput. This is because flexible Paxos talks to all nodes and takes the first quorum of responses for satisfying the phase-2 requirement, although the other responses also arrive and overwhelm the leader. While it is possible to use a thrifty optimization [36, 41] and contact only the quorum of nodes for phase-2, in that case, a single faulty or sluggish node may stall the performance. In contrast to the flexible quorum approach, PigPaxos clears the bottleneck at the leader while also preserving robustness against failures. However, it is possible to employ the PigPaxos approach with the flexible quorums idea, for providing improved scalability and throughput to FPaxos [29].

WPaxos [3] presents a WAN-optimized multi-leader Flexible Paxos[29] protocol. WPaxos selects per-object leaders and employs an object stealing protocol with adaptive stealing improvements to match the workload access locality [13]. More specifically, concurrent leaders from different zones may steal ownership of objects from each other using phase-1 of Paxos. They use small phase-2 to commit operations on these objects locally until the ownership gets stolen by other leaders. The multi-leader, multi-quorum setup in WPaxos helps with both WAN latency and throughput due to the smaller and geographically localized quorums. In contrast to WPaxos, PigPaxos improves throughput and scalability using a single leader. However, PigPaxos optimization is still applicable to WPaxos. For instance, WPaxos leaders can employ Pig to fully replicate to many nodes and learn when these additional nodes commit. This technique could be useful for achieving bounded-staleness [1] at large-scale geo-replicated database deployments.

Mencius [6] uses rotating leaders to alleviate the bottleneck at a single leader to improve throughput. For Mencius to work effectively, the workload has to be partitioned across each leader nicely, so that each leader has something to propose in their turn. Stalls in rotating leader proposals create problems with Mencius performance, and delays and failures cause tricky corner cases. In contrast, PigPaxos is simple to implement and it avoids these issues. Instead of rotating leaders, PigPaxos rotates the small number of relay nodes the leader interacts with. This clears the communication bottlenecks and improves scalability and throughput, while still keeping the protocol simple and avoiding the complexity and performance complications Mencius suffers from.

Canopus [46] is a multi-leader consensus protocol aimed at wide-area deployments. It trades off latency and fault-tolerance for throughput. The protocol heavily relies on batching and requires many communication steps for a single consensus round. Canopus divides the nodes into a fixed number of groups, called

super-leaves. Each node within the super-leaf exchanges the list of proposed commands with other super-leaf peers using the reliable broadcast primitive. Nodes within a super-leaf then order these commands in the same deterministic order, creating a virtual node that combines the information of the entire super-leaf. The process repeats, but now with the virtual nodes grouped together and exchanging the commands until only one root virtual node remains. At that point, every physical node has all commands in the same order. Canopus suffers from fault-tolerance problems, as it cannot tolerate a network partition or failure of any single super-leaf. Since the nodes that constitute a super-leaf need to be located within the same rack (for using the reliable broadcast primitive), Canopus is especially prone to correlated failures and loss of availability. In contrast to Canopus, PigPaxos can tolerate failures of up to half of the nodes in the system. Moreover, in most cases, PigPaxos achieves high-throughput without an observable increase in latency.

EPaxos [41], and Atlas [19] try to opportunistically commit commands. Any node in these systems becomes an opportunistic leader for command and tries to commit it by running a phase-2 of Paxos in a super-majority quorum system. If some other node in the super-majority quorum is also working on a conflicting command, then some conflict resolution must happen to order the operations with respect to each other. Often this conflict resolution involves falling back to Paxos. These protocols suffer from an increased number of conflicting commands for heavy workloads, and their performance plummets. In contrast to EPaxos, PigPaxos provides linearizability of all operations and achieves high throughput within a more simple single leader Paxos implementation.

Compartmentalized Paxos [54], like a classical single instance Paxos [35], breaks down Paxos protocol into multiple distinct roles and allocates these roles to separate nodes. The protocol introduces proxy-leaders, who are responsible for tallying votes and acquiring the majority of votes on the proposers' behalf. The stable proposer itself is only responsible for establishing the order of operations and selecting a different proxy-leader for each command. Proxy-leaders, in a similar manner, talk to flexible quorum acceptors to commit operations. The proxy-leader notion is somewhat similar to a single relay group setup in PigPaxos, however, our approach does not separate the roles, and delegates fewer responsibilities to the relays. Similar to PigPaxos, the compartmentalized idea introduces additional failure modes, and recovers by retries.

7.2 Weakly consistent replication

Saito et al. [47] provide a comprehensive survey of the weakly consistent replication protocols that promise higher availability and performance, but they allow replicas to temporarily diverge and let users see inconsistent data. In these techniques, updates are propagated in the background, and occasional conflicts are fixed after they happen. Since they require little synchronization among replicas, they can scale to a large number of replicas. Weakly consistent replication approaches provide only bounded-divergence or eventual consistency guarantees, rather than strong consistency guarantees. Weak consistency replication protocols use multiple concurrent leaders to circumvent the single leader bottlenecks. For communication, they use flooding, rumor-mongering, and directed gossiping. While the Pig communication primitive could also provide some performance improvements to weak consistency replication protocols, those would not be as pronounced.

7.3 Blockchain protocols

Blockchain protocols solve consensus in the presence of Byzantine nodes. Recent work on blockchain consensus protocols employ a tiered hybrid method, and choose a council among all participants [22, 33] to reduce the size of consensus quorums, and then run Byzantine fault-tolerant protocols to achieve quick finality in a deterministic manner in this council. In particular, the Practical Byzantine Fault Tolerance (PBFT) [11] protocol forms a basis/template for most Byzantine fault-tolerant (BFT) consensus protocols, such as Casper [10] used in Ethereum, Tendermint [8] used in Cosmos, and HotStuff [55] used in Facebook's Libra. Since these protocols have similar protocol architecture to Paxos, the PigPaxos aggregation technique we introduced here can also be adopted to help improve scalability for these protocols.

Several blockchain protocols [23, 38] use gossip-based message dispersal instead of point-to-point communication. FastBFT [38] introduces a novel message aggregation technique that combines hardware-based trusted execution environments with lightweight secret sharing, and reduces the BFT consensus message complexity from O(n^2) to O(n). FastBFT arranges nodes in a tree topology, and imposes that inter-server communication and message aggregation take place along edges of the tree. Scalable Byzantine Reliable Broadcast [23] presents a probabilistic gossip-based Byzantine reliable broadcast algorithm that uses samples as a replacement for quorums, and introduces threshold contagion model of message propagation. In contrast to these work, our work emphasized the use of relaying for control message dissemination for crash fault-tolerance consensus protocols and improves the $O(N)$ cost further, by using a simple dynamic rotated aggregation technique and operating in a deterministic fashion.

Similar to the crash fault-tolerant consensus protocols, in the BFT consensus domain, multi-leader approaches [25, 48] have been presented to alleviate the bottleneck at a single-leader. Mir-BFT [48] generalizes PBFT and allows a set of leaders to propose request batches independently, and rotates the assignment of a partitioned request hash space to leaders. To achieve multi-leader BFT consensus, Gupta et. al. propose running several instances of consensus protocol in parallel and present coordination-free techniques to order operations across parallel instances. Finally, work on adopting BFT consensus protocols to WAN deployments [4, 26] also make use of region based grouping and aggregation of communication.

8 FUTURE WORK AND EXTENSIONS

8.1 Dynamic & Overlapping Relay Groups

In the basic scheme described in Section 3, PigPaxos leader randomly selects a node from every relay group. The relay groups themselves are static and pre-configured. It is possible to change the configuration of relay groups on-the-fly to improve the performance or to react quickly to crashes in some of the groups. Consider a scenario where a particular configuration of relay groups starts to experience degradation in performance. In this case, the leader may reshuffle the slower nodes into some minority relay group to improve the performance. The leader may specify a new relay group

for each round of the protocol by including future group membership information or a seed value to deterministically compute such membership in every sent message.

Another relay group optimization is to allow groups to overlap. Although this decreases efficiency due to increasing the number of messages sent and received, having alternate paths to reach nodes will improve the reliability and performance predictability in environments with link volatility and failures. With the group overlap, some follower nodes may be part of more than one relay group, creating a possibility that a node casts its vote twice. However, PigPaxos remains safe, since the leader keeps track of all followers acknowledging the command, allowing the leader to catch a node casting more than one vote.

8.2 Improving Reads

Reading from Paxos-backed state machines is traditionally performed in one of three different ways: reading from any node, reading from the leader node, or reading by the virtue of serializing the read operation in the log. Unfortunately, each of these methods has some drawbacks. Reading from any replica, while fast, compromises the strong consistency guarantee, since the state at a replica may be stale [15]. Reading from a "stable" leader requires additional mechanisms, such as leader leases in order to prevent "split-brain" when multiple leader candidates emerge. Finally, performing the read by serializing the read operation on the log incurs latency and throughput costs since it requires a full Paxos round to complete. Recent work on Paxos Quorum Reads (PQR) [14] provides an alternative method for strongly consistent reads. In PQR, reads are performed by the clients directly contacting the underutilized replicas, avoiding the leader, and improving the throughput.

A client multicasts the read request to a read quorum of replicas and waits for their replies. The read is concluded if a read quorum of replicas returns the same slot number for the last accepted and applied slots of the requested data item. Otherwise, a second phase of read is required to confirm that the higher accepted slot is applied and that value is returned as the result.

PigPaxos is especially synergistic with PQR because, due to its efficient dynamic relay based aggregation mechanism, it can compensate using a larger write quorum and allow for a small-sized read quorum. Small read quorums imply that multiple read quorums can be utilized in parallel by the clients to perform the read directly with the replicas. This helps in scaling the read throughput without bogging down the leader.

The Pig communication pattern itself can also be employed for PQR reads. A read-heavy client can benefit from employing randomly rotated pig relay nodes to form aggregation trees to perform the PQR read. If it could be possible for the client to know about the schedule of the leader's relay nodes, it can optimize the PQR reads further by tapping into the leader's relay nodes and performing PQR using the aggregated information at the relay nodes instead of contacting the leaf replicas.

8.3 Scaling to Large Clusters

One direction for future work is to design and evaluate further optimization opportunities in the communication tree of PigPaxos, and to scale the protocol to hundreds of nodes. This is likely to open many new challenges. When scaled to two-orders of magnitude, we can argue that the transformation is not incremental/evolutionary and would necessitate new supplementary approaches and services for coping with this scale. For example, in the Paxos failure detector module, the threshold for a node to become a leader candidate should be modified to be conservatively high, as contesting the leader is very costly at the scale of hundred participants. Therefore, instead of static failure detector thresholds, new approaches are needed to set the thresholds to be proportional to the size of N, such that only a couple nodes at a given time period will step up to become leader candidates.

8.4 Applying Pig to Other Protocols

While we have applied Pig to Multi-Paxos style of protocols, it may be helpful in many other situations requiring a fan-out/fan-in style of communication at a large scale. For instance, some large planetary-scale and edge systems, such as WPaxos [3] and DPaxos [42], perform phase-1 of Paxos across a large set of nodes for initial data placement or ownership changes. If such phase-1 occurs frequently, for example, during data ingestion or migration between regions, it may put more load on the servers, impacting both the phase-1 operation and regular replication. Applying Pig in this situation can help alleviate the issues and improve the performance of certain workloads in such large geo-scale deployments.

Another future work item is to apply and integrate our approach for achieving scalability to modern PBFT [11] descendant byzantine fault-tolerant blockchain protocols, such as Tendermint [8], Casper [10], and LibraBFT [7, 55].

9 CONCLUDING REMARKS

To address the communication bottlenecks at strong consistency replication protocols, we presented the Pig communication primitive. Pig manages to reduce the leader's communication bottleneck by decoupling the decision-making at the leader from the communication at the leader. Although aggregation approaches have been known and employed in the context of weak-consistency replication protocols, Pig shows how they can be effectively applied to and integrated with the strong consistency distributed consensus protocols, such as Paxos.

Our evaluations show that PigPaxos can provide more than 3 folds improvement in throughput over Paxos and EPaxos with little latency deterioration. Our experiments also show that the aggregation used in PigPaxos has little latency overhead as compared to the latency of Paxos. The communication piggybacking and aggregation technique used in PigPaxos is readily applicable to many Paxos implementations [43, 44] and variants [3, 29], and we conjecture that PigPaxos would be useful for implementing geo-replicated distributed databases with tens of replicas distributed over many regions around the globe.

ACKNOWLEDGMENTS

This project is in part sponsored by the National Science Foundation (NSF) under award number CNS-2008243.

REFERENCES

[1] M. K. Aguilera and D. B. Terry. The many faces of consistency. *IEEE Data Eng. Bull.*, 39(1):3–13, 2016.

[2] A. Ailijiang, A. Charapko, and M. Demirbas. Dissecting the performance of strongly-consistent replication protocols. In *Proceedings of the 2019 International Conference on Management of Data*, pages 1696–1710. ACM, 2019.

[3] A. Ailijiang, A. Charapko, M. Demirbas, and T. Kosar. WPaxos: Wide area network flexible consensus. *IEEE Transactions on Parallel and Distributed Systems*, 31(1):211–223, 2019.

[4] Y. Amir, C. Danilov, D. Dolev, J. Kirsch, J. Lane, C. Nita-Rotaru, J. Olsen, and D. Zage. Steward: Scaling byzantine fault-tolerant replication to wide area networks. *IEEE Transactions on Dependable and Secure Computing*, 7(1):80–93, 2008.

[5] J. Baker, C. Bond, J. Corbett, J. Furman, A. Khorlin, J. Larson, J. Léon, Y. Li, A. Lloyd, and V. Yushprakh. Megastore: Providing scalable, highly available storage for interactive services. *CIDR*, pages 223–234, 2011.

[6] C.-S. Barcelona. Mencius: building efficient replicated state machines for wans. *8th {USENIX} Symposium on Operating Systems Design and Implementation (OSDI 08)*, 2008.

[7] M. Baudet, A. Ching, A. Chursin, G. Danezis, F. Garillot, Z. Li, D. Malkhi, O. Naor, D. Perelman, and A. Sonnino. State machine replication in the libra blockchain, 2019.

[8] E. Buchman. *Tendermint: Byzantine fault tolerance in the age of blockchains.* PhD thesis, 2016.

[9] M. Burrows. The chubby lock service for loosely-coupled distributed systems. In *Proceedings of the 7th {USENIX} Symposium on Operating Systems Design and Implementation*, pages 335–350. USENIX Association, 2006.

[10] V. Buterin and V. Griffith. Casper the friendly finality gadget. 2017.

[11] M. Castro and B. Liskov. Practical byzantine fault tolerance. In *Proceedings of the Third Symposium on Operating Systems Design and Implementation*, OSDI '99, pages 173–186, USA, 1999. USENIX Association.

[12] T. D. Chandra, R. Griesemer, and J. Redstone. Paxos made live: an engineering perspective. In *Proceedings of the twenty-sixth annual ACM symposium on Principles of distributed computing*, pages 398–407. ACM, 2007.

[13] A. Charapko, A. Ailijiang, and M. Demirbas. Adapting to access locality via live data migration in globally distributed datastores. In *2018 IEEE International Conference on Big Data (Big Data)*, pages 3321–3330. IEEE, 2018.

[14] A. Charapko, A. Ailijiang, and M. Demirbas. Linearizable quorum reads in Paxos. In *11th {USENIX} Workshop on Hot Topics in Storage and File Systems (HotStorage 19)*, 2019.

[15] A. Charapko, A. Ailijiang, M. Demirbas, and S. Kulkarni. Retroscope: Retrospective monitoring of distributed systems. *IEEE Transactions on Parallel and Distributed Systems*, 30(11):2582–2594, Nov 2019.

[16] Cockroach Labs. Cockroachdb: The sql database for global cloud services. https://www.cockroachlabs.com/docs/stable/architecture/overview.html, 2018.

[17] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears. Benchmarking cloud serving systems with ycsb. In *Proceedings of the 1st ACM symposium on Cloud computing*, pages 143–154, 2010.

[18] J. Corbett, J. Dean, M. Epstein, A. Fikes, C. Frost, J. Furman, S. Ghemawat, A. Gubarev, C. Heiser, P. Hochschild, W. Hsieh, S. Kanthak, E. Kogan, H. Li, A. Lloyd, S. Melnik, D. Mwaura, D. Nagle, S. Quinlan, R. Rao, L. Rolig, Y. Saito, M. Szymaniak, C. Taylor, R. Wang, and D. Woodford. Spanner: Google's globally-distributed database. *Proceedings of OSDI*, 2012.

[19] V. Enes, C. Baquero, T. F. Rezende, A. Gotsman, M. Perrin, and P. Sutra. State-machine replication for planet-scale systems. In *Proceedings of the Fifteenth European Conference on Computer Systems*, EuroSys '20, pages 1–15, 2020.

[20] Facebook, Inc. Delos: Simple, flexible storage for the Facebook control plane. https://engineering.fb.com/data-center-engineering/delos/, 2019.

[21] M. J. Fischer, N. A. Lynch, and M. S. Paterson. Impossibility of distributed consensus with one faulty process. *J. ACM*, 32(2):374–382, Apr. 1985.

[22] Y. Gilad, R. Hemo, S. Micali, G. Vlachos, and N. Zeldovich. Algorand: Scaling byzantine agreements for cryptocurrencies. In *Proceedings of the 26th Symposium on Operating Systems Principles*, pages 51–68, 2017.

[23] R. Guerraoui, P. Kuznetsov, M. Monti, M. Pavlovic, D.-A. Seredinschi, and Y. Vonlanthen. Scalable byzantine reliable broadcast. *DISC*, 2019.

[24] H. S. Gunawi, M. Hao, T. Leesatapornwongsa, T. Patana-anake, T. Do, J. Adityatama, K. J. Eliazar, A. Laksono, J. F. Lukman, V. Martin, and A. D. Satria. What bugs live in the cloud? a study of 3000+ issues in cloud systems. In *Proceedings of the ACM Symposium on Cloud Computing*, SOCC '14, pages 1–14, New York, NY, USA, 2014. Association for Computing Machinery.

[25] S. Gupta, J. Hellings, and M. Sadoghi. Revisiting consensus protocols through wait-free parallelization. *arXiv preprint arXiv:1908.01458*, 2019.

[26] S. Gupta, S. Rahnama, J. Hellings, and M. Sadoghi. Resilientdb: Global scale resilient blockchain fabric. *arXiv preprint arXiv:2002.00160*, 2020.

[27] M. P. Herlihy and J. M. Wing. Linearizability: A correctness condition for concurrent objects. *ACM Trans. Program. Lang. Syst.*, 12(3):463–492, July 1990.

[28] E. N. Hoch, Y. Ben-Yehuda, N. Lewis, and A. Vigder. Bizur: A key-value consensus algorithm for scalable file-systems. *arXiv preprint arXiv:1702.04242*, 2017.

[29] H. Howard, D. Malkhi, and A. Spiegelman. Flexible paxos: Quorum intersection revisited. *arXiv preprint arXiv:1608.06696*, 2016.

[30] P. Hunt, M. Konar, F. P. Junqueira, and B. Reed. ZooKeeper: Wait-free coordination for internet-scale systems. In *{USENIX} Annual Technical Conference ({USENIX} ATC'10)*, volume 8. Boston, MA, USA, 2010.

[31] F. P. Junqueira, B. C. Reed, and M. Serafini. Zab: High-performance broadcast for primary-backup systems. In *2011 IEEE/IFIP 41st International Conference on Dependable Systems & Networks (DSN)*, pages 245–256. IEEE, 2011.

[32] K. Kingsbury. Jepsen tests, 2017. https://jepsen.io/.

[33] E. K. Kogias, P. Jovanovic, N. Gailly, I. Khoffi, L. Gasser, and B. Ford. Enhancing bitcoin security and performance with strong consistency via collective signing. In *25th {USENIX} security symposium ({USENIX} security 16)*, pages 279–296, 2016.

[34] J. Kreps, N. Narkhede, J. Rao, et al. Kafka: A distributed messaging system for log processing. NetDB, 2011.

[35] L. Lamport. Paxos made simple. *ACM SIGACT News*, 32(4):18–25, 2001.

[36] L. Lamport and M. Massa. Cheap paxos. In *International Conference on Dependable Systems and Networks, 2004*, pages 307–314. IEEE, 2004.

[37] T. Leesatapornwongsa, J. F. Lukman, S. Lu, and H. S. Gunawi. Taxdc: A taxonomy of non-deterministic concurrency bugs in datacenter distributed systems. *SIGPLAN Not.*, 51(4):517–530, Mar. 2016.

[38] J. Liu, W. Li, G. O. Karame, and N. Asokan. Scalable byzantine consensus via hardware-assisted secret sharing. *IEEE Transactions on Computers*, 68(1):139–151, 2018.

[39] M. Marchukov. LogDevice: a distributed data store for logs. https://engineering.fb.com/core-data/logdevice-a-distributed-data-store-for-logs/, 2017.

[40] Microsoft. Cosmos DB - globally distributed, multi-model database service. https://docs.microsoft.com/en-us/azure/cosmos-db/global-dist-under-the-hood, 2018.

[41] I. Moraru, D. G. Andersen, and M. Kaminsky. There is more consensus in egalitarian parliaments. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*, pages 358–372. ACM, 2013.

[42] F. Nawab, D. Agrawal, and A. El Abbadi. DPaxos: Managing data closer to users for low-latency and mobile applications. In *Proceedings of the 2018 International Conference on Management of Data*, pages 1221–1236. ACM, 2018.

[43] D. Ongaro and J. Ousterhout. In search of an understandable consensus algorithm. In *2014 {USENIX} Annual Technical Conference ({USENIX} ATC 14)*, pages 305–319, 2014.

[44] Red Hat. etcd. a distributed, reliable key-value store for the most critical data of a distributed system. https://coreos.com/etcd/, 2019.

[45] K. Ren, D. Li, and D. J. Abadi. Slog: Serializable, low-latency, geo-replicated transactions. *Proc. VLDB Endow.*, 12(11):1747–1761, July 2019.

[46] S. Rizvi, B. Wong, and S. Keshav. Canopus: A scalable and massively parallel consensus protocol. In *Proceedings of the 13th International Conference on Emerging Networking EXperiments and Technologies*, CoNEXT '17, pages 426–438, New York, NY, USA, 2017. Association for Computing Machinery.

[47] Y. Saito and M. Shapiro. Optimistic replication. Technical Report MSR-TR-2003-60, October 2003.

[48] C. Stathakopoulou, T. David, and M. Vukolić. Mir-BFT: High-throughput bft for blockchains. *arXiv preprint arXiv:1906.05552*, 2019.

[49] C. Tang, T. Kooburat, P. Venkatachalam, A. Chander, Z. Wen, A. Narayanan, P. Dowell, and R. Karl. Holistic configuration management at Facebook. In *Proceedings of the 25th Symposium on Operating Systems Principles*, SOSP '15, pages 328–343, New York, NY, USA, 2015. ACM.

[50] The Apache Software Foundation. Apache Cassandra. http://cassandra.apache.org, 2019.

[51] R. van Renesse and D. Altinbuken. Paxos made moderately complex. *ACM Comput. Surv.*, 47(3), Feb. 2015.

[52] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes. Large-scale cluster management at google with borg. In *Proceedings of the Tenth European Conference on Computer Systems*, page 18. ACM, 2015.

[53] W. Vogels. Eventually consistent. *Communications of the ACM*, 52(1):40–44, 2009.

[54] M. Whittaker, A. Ailijiang, A. Charapko, M. Demirbas, N. Giridharan, J. M. Hellerstein, H. Howard, I. Stoica, and A. Szekeres. Scaling replicated state machines with compartmentalization [technical report], 2021.

[55] M. Yin, D. Malkhi, M. K. Reiter, G. G. Gueta, and I. Abraham. HotStuff: BFT consensus with linearity and responsiveness. In *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing*, PODC '19, pages 347–356, New York, NY, USA, 2019. Association for Computing Machinery.

[56] Yugabyte, Inc. YugabyteDB. https://www.yugabyte.com, 2020.

[57] J. Zheng, Q. Lin, J. Xu, C. Wei, C. Zeng, P. Yang, and Y. Zhang. PaxosStore: high-availability storage made practical in WeChat. *Proceedings of the VLDB Endowment*, 10(12):1730–1741, 2017.